

Практическая работа №7

Теоретическое введение

GitLab — веб-инструмент жизненного цикла DevOps с открытым исходным кодом, представляющий систему управления репозиториями кода для Git с собственной вики, системой отслеживания ошибок, CI/CD пайплайном и другими функциями.

GitLab Runner — это агент, который собственно и занимается выполнением инструкций из специального файла `.gitlab-ci.yml`.

Пайплайн (от английского pipeline — «трубопровод») — это документ, визуализирующий процесс разработки продукта. Он представляет собой последовательность этапов, расположенных так, что конец предыдущего является началом следующего. Благодаря этому создается эффект производственного конвейера или трубопровода, по которому проект движется от первоначальной идеи до конкретного продукта.

Полезные ссылки

1. Терминология. Виртуальные машины [Несколько компьютеров в одном]:

<https://it.wikireading.ru/326>

2. Глоссарий | ITGLOBAL.COM:

<https://itglobal.com/ru-ru/company/glossary/>

3. YAML для начинающих:

https://1cloud.ru/blog/yaml_for_beginners

Задание

В данной практической работе требуется произвести цикл настройки CI/CD в GitLab.

Ниже представлен пример настройки цикла CI/CD проекта на Maven.

Требуется создать в корне проекта файл `.gitlab-ci.yml`. Имеется возможность указать нужный образ, вместо дефолтного образа у раннера.

```
image: maven:latest
```

Нужно установить место хранения папки `.m2` через переменные среды `variables`. Это понадобится, когда будет настраиваться кэширование.

```
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=./.m2/repository"
```

Далее указываются этапы сборки. Они позволяют группировать задачи для одновременного выполнения.

```
stages:
```

- build
- test
- package
- deploy
- notify

- build – стадия сборки.
- test – стадия тестирования.
- package – стадия упаковки в jar.
- deploy – стадия деплоя на сервер.
- notify – стадия уведомления о провале.

Далее производится указание задач для каждой стадии.

Сборка - Build

```
build:
```

```
stage: build
only:
  - dev
  - merge_requests
  - /^release\/.*$/
except:
  - tags
script:
  - 'mvn --settings $MAVEN_SETTINGS compile'
cache:
  paths:
    - ./target
    - ~/.m2
```

Раздел script выполняет linux команды.

Переменная GitLab CI/CD \$MAVEN_SETTINGS необходима для передачи файла settings.xml, если используются нестандартные настройки, например корпоративные репозитории.

Переменная создается в настройках CI/CD для группы/проекта. Тип переменной File.

Раздел only указывает для каких веток и тегов выполнять задачу. Чтобы не собирать каждую запущенную ветку требуется установить: dev, merge_requests и ветки формата /release/*.

Раздел only не отличает ветки от тегов. Поэтому указывается параметр except, который исключает теги. Из-за этого поведения за сборку на прод отвечают отдельные задачи. В противном случае, если бы кто-то создал тег формата release/, то он бы запустил сборку.

Для защиты от случайного деплоя в прод, рекомендуется установить защиту на ветки dev, master, /release/*, а так же на теги release-*. Делается это в настройках проекта GitLab.

Раздел cache отвечает за кэширование. Чтобы каждый раз не выкачивать зависимости требуется добавить в кэш папку ./target и ~/.m2.

Запуск unit тестов – test

Далее производится создание задачи для запуска тестов.

```

test:
  stage: test
  only:
    - dev
    - merge_requests
    - /^release\/.*$/
  except:
    - tags
  script:
    - 'mvn --settings $MAVEN_SETTINGS test'
  cache:
    paths:
      - ./target
      - ./m2

```

Ничем не отличается от стадии сборки, только команда мавена test. Кэш необходимо указать и в этой задаче, чтобы получить его и отправить дальше.

Упаковка – package

Следом добавляется задача упаковки с выключенными тестами. Она уже выполняется только для веток dev и release/*. Упаковывать Merge Request смысла нет.

```

package:
  stage: package
  only:
    - dev
    - /^release\/.*$/
  except:
    - tags
  script:
    - 'mvn --settings $MAVEN_SETTINGS package -
Dmaven.test.skip=true'
  artifacts:
    paths:
      - target/*.jar
  cache:
    policy: pull
    paths:
      - ./target
      - ./m2

```

Параметр `policy: pull` загрузит кэш для этой задачи, но не отправит его для следующей.

Для сохранения результатов сборки используется `artifacts`, который сохраняет `.jar` файлы и передает их следующим задачам.

Деплой – `deploy`

Деплой будет производиться с помощью переноса `.jar` файла на сервер.

Перенос `jar` на сервер с помощью стандартных `ssh` и `scp`.

```
deploy_dev_server:
  stage: deploy
  only:
    - dev
  except:
    - tags
  before_script:
    - which ssh-agent || ( dnf update -y && dnf install openssh-
client -y )
    - eval $(ssh-agent -s)
    - echo "$SSH_PRIVATE_KEY" | ssh-add -
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
    - ssh-keyscan $DEV_HOST >> ~/.ssh/known_hosts
    - chmod 644 ~/.ssh/known_hosts
  script:
    - ssh $DEV_USER@$DEV_HOST "[ ! -f $DEV_APP_PATH/app_name.jar
] || mv $DEV_APP_PATH/app_name.jar $DEV_APP_PATH/app_name-
build-$CI_PIPELINE_ID.jar"
    - scp target/app_name.jar
$DEV_USER@$DEV_HOST:$DEV_APP_PATH/
    - ssh $DEV_USER@$DEV_HOST "sudo systemctl stop
app_name_service && sudo systemctl start app_name_service"
```

Требуется создать переменные в GitLab CI CD:

- `$DEV_USER` – пользователь системы, от чьего имени будет происходить деплой.
- `$DEV_HOST` – `ip` адрес сервера.
- `$DEV_APP_PATH` – путь до папки приложения.

- `SSH_PRIVATE_KEY` – приватный ключ.

Раздел `before_script` отвечает за выполнение настроек перед основным скриптом. Производится проверка на наличие `ssh-agent`, устанавливаем его при отсутствии. После чего добавляем приватный ключ, устанавливаем правильные права на папки.

В разделе `script` происходит деплой на сервер:

1. Проверяем наличие старого `jar` и переименовываем его.
`SCI_PIPELINE_ID` - это глобальный номер сборки Pipeline.
2. Копируем новый `jar` на сервер.
3. Останавливаем и запускаем службу, отвечающую за приложение.

На пре-прод делаем по аналогии, только меняем переменные на `$PRE_PROD_*`.

```
deploy_pre_prod:
  stage: deploy
  only:
    - /^release\/.*$/
  except:
    - tags
  before_script:
    - which ssh-agent || ( dnf update -y && dnf install openssh-
      client -y )
    - eval $(ssh-agent -s)
    - echo "$SSH_PRIVATE_KEY" | ssh-add -
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
    - ssh-keyscan $PRE_PROD_HOST >> ~/.ssh/known_hosts
    - chmod 644 ~/.ssh/known_hosts
  script:
    - ssh $PRE_PROD_USER@$PRE_PROD_HOST "[ ! -f
      $PRE_PROD_APP_PATH/app_name.jar ] || mv
      $PRE_PROD_APP_PATH/app_name.jar $PRE_PROD_APP_PATH/app_name-
      build-$SCI_PIPELINE_ID.jar"
    - scp target/app_name.jar
      $DEV_USER@$PRE_PROD_HOST:$PRE_PROD_APP_PATH/
    - ssh $PRE_PROD_USER@$PRE_PROD_HOST "sudo systemctl stop
      app_name_service && sudo systemctl start app_name_service"
```

Настройка деплоя на прод

Для прод-деплоя можно объединить сборку, тестирование и упаковку в одну задачу.

```
package_prod:
  stage: package
  only:
    - /^release-.*$/
  except:
    - branches
  script:
    - 'mvn --settings $MAVEN_SETTINGS package'
  artifacts:
    paths:
      - target/*.jar
```

Деплой аналогичен пре-проду, изменяется только `only` и переменные.

Дополнительно появляется задача с отправкой артефакта в корпоративный `nexus`. Деплой на сервер и в нексус работает параллельно.

```
deploy_prod:
  stage: deploy
  only:
    - /^release-.*$/
  except:
    - branches
  ...

deploy_nexus_server:
  stage: deploy
  only:
    - /^release-.*$/
  except:
    - branches
  script:
    - 'mvn --settings $MAVEN_SETTINGS deploy -
Dmaven.test.skip=true'
```

В результате отчет должен содержать титульный лист, скриншоты процесса цикла настройки CI/CD в GitLab, а также ответы на контрольные вопросы.

Вопросы к практической работе

1. Назовите ключевые особенности GitLab и дайте им описание.
2. Опишите различия GitLab CI/CD и GitHub Actions.
3. Дайте описание готовым шаблонам, которые можно взять за основы при написании своего сценария.
4. Приведите пример общего синтаксиса опций, которые применяются при описании задания.
5. Дайте описание опции Stage, которая применяется при описании задания.
6. Дайте описание опции Image, которая применяется при описании задания.

Критерии оценки

- Произведен цикл настройки CI/CD в GitLab
- Продемонстрирована работоспособность
- В отчете представлены ответы на контрольные вопросы